OPEN SOURCE MIT-0

Agentic Coding Harness & Benchmarks

Cut agentic-coding **token spend** by choosing the harness and model from a measured **cost/quality frontier** -- on Amazon Bedrock or self-hosted.

CLAUDE CODE & PI AMAZON BEDROCK SELF-HOSTED VLLM LLM-AS-JUDGE

Practitioner + executive briefing

What we'll cover

01 The problem: agentic-coding token spend

02 The approach & the headline result

03 Reading the frontier: models & harness choice

04 How cost is measured & self-hosted economics

05 Get started

A quick read for executives, with practitioner and methodology detail as you go deeper.

01

The problem: token spend

Every developer and every task adds to the bill. The two biggest levers -- which harness, which model -- are usually chosen on gut feel.

Agentic coding burns tokens at scale

Long-horizon, not one-shot

A task is tens to hundreds of LLM turns (~50-250, up to 300+), running 10 minutes to over an hour -- not a single prompt/response.

Prefill-heavy shape

Each turn replays a growing transcript as fresh input and emits a tiny edit. Real input:output runs **~150:1 to ~660:1** -- not the ~3:1 generic pricing assumes.

Cost hides in the choices

How many turns, how much context re-read, which harness, which model -- these swing the bill several-fold for similar quality.

Organizations are already at -- or heading toward -- trillions of tokens per month, and coding agents drive a large and growing share of it. That is the bill this repo helps control.

Public leaderboards can be saturated, and generic token pricing does not reflect this workload -- you need numbers on work that looks like yours.

02

The approach & the headline

Measure quality and cost on real tasks, across harnesses and hosting paths, then let the Pareto frontier guide the choice.

Two benchmarks, combined over real tasks

Quality

How well a model does a real coding task -- scored 0-100 by an independent LLM judge (**codex exec, GPT-5.6-sol**, high reasoning effort) against a fixed rubric.

Throughput → cost

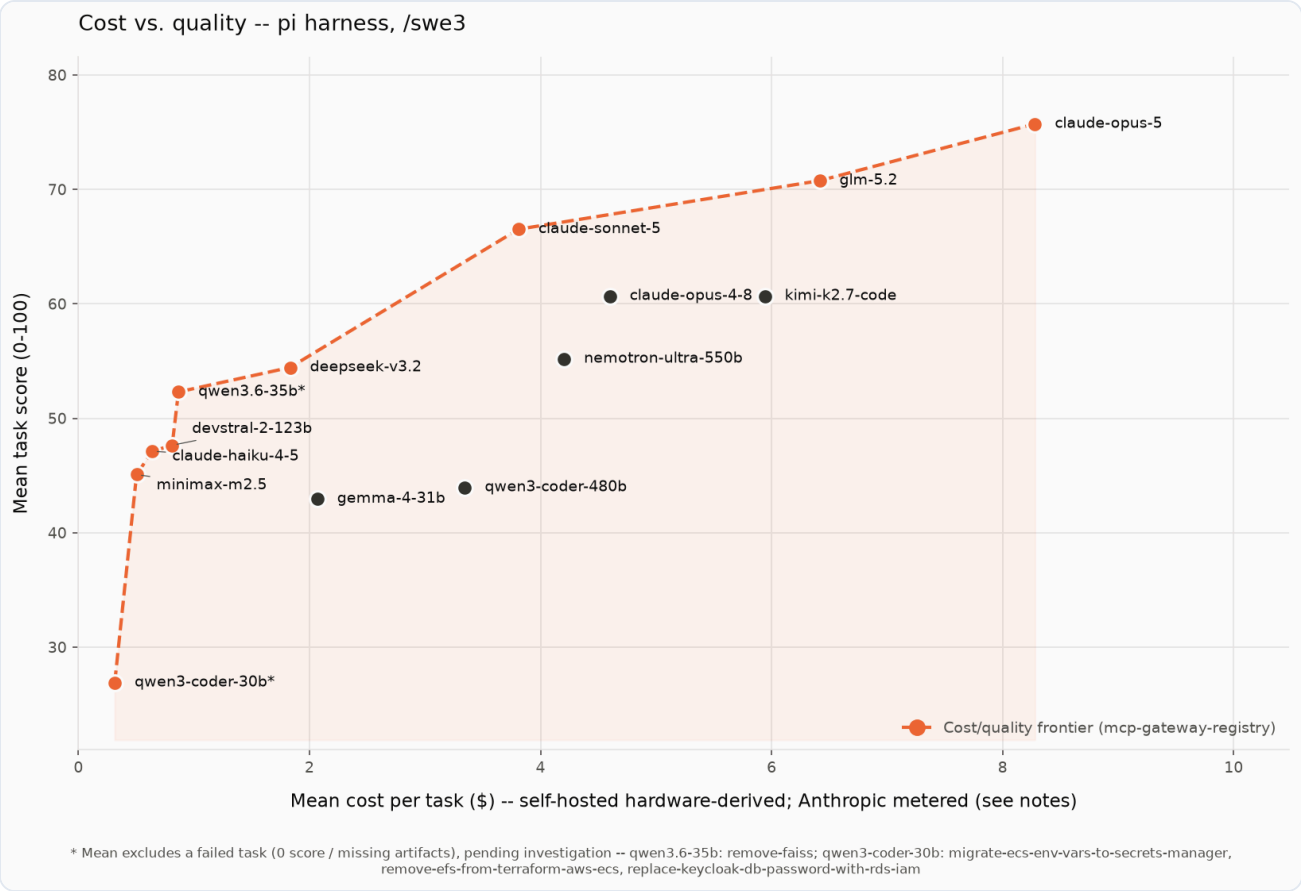
Tokens/sec a self-hosted model sustains on a GPU, turning the instance's hourly price into a hardware-derived cost per task.

= Pareto frontier

Plot cost vs quality: the models where nothing else is both better and cheaper. That is the buying guide.

Crossing **harnesses** (Claude Code, pi) with **models** across **three hosting paths** gives real optionality -- the same model can be pricier under one agent yet cheaper and faster under another.

The cost/quality frontier (pi, /swe3)



14 models, 5 real tasks on `mcp-gateway-registry`. x = mean cost/task, y = mean score. The dashed line is the non-dominated frontier. Anthropic points are metered Bedrock bills; self-hosted points are hardware-derived (different bases -- compare within a hosting basis).

What the frontier tells a buyer

75.7

claude-opus-5

Top quality, $8.28/task on pi -- the premium tier for business-critical work.

70.8

glm-5.2

Best open-weight model; the self-hosted quality anchor.

3×

cheaper on pi

opus-5 under pi vs Claude Code: higher score, ~1/3 the cost, half the wall-clock.

Same model, right harness = large cost win for little or no quality give-up. Pick the model for the job; run it on the harness the frontier favors.

PRACTITIONER DETAIL

03

Reading the frontier

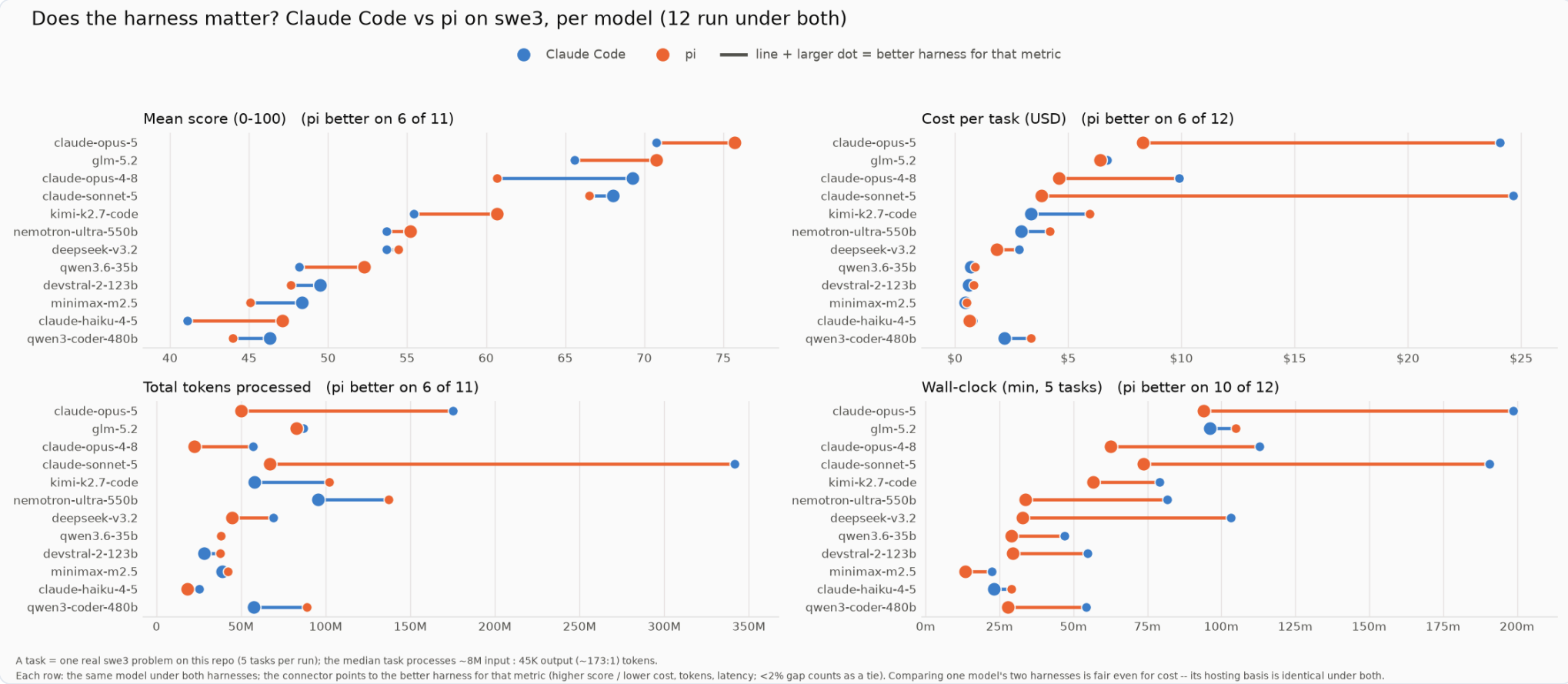
Which model for which job, and which harness to drive it with. Skip ahead if you only needed the headline.

Pick by tier, on pi (numbers are /swe3)

Premium `claude-opus-5`	Business-critical / high-stakes changes. Highest quality (75.7) at $8.28/task; wrong designs are expensive here.
Premium open-weight `glm-5.2`	Best self-hosted quality (70.8, $6.42/task). Standardize on it if you self-host your frontier tier.
Reliable workhorse `deepseek-v3.2`	Bulk of day-to-day self-hosted coding: 54.4 at $1.83/task, completes 5/5 -- most of the quality for a fraction of the cost.
Most cost-effective `claude-haiku-4-5`	Boilerplate, small fixes, scaffolding you'll review anyway: 47.1 at $0.64/task.

A model that is merely cheap but *dominated* (off the frontier) -- or that does not reliably finish -- is not a bargain.

Does the harness matter? Yes.



Same model, both harnesses, per metric. The connector points to the better harness; each panel tallies how often pi wins.

pi wins wall-clock on 10 of 12 models and ties or wins on cost -- its single-agent loop skips the sub-agent fan-out that inflates Claude Code's tokens, dollars, and time.

Pick the harness per model -- but pi is the default

Metric (12 models, both harnesses)	pi wins	Claude Code wins	tie
Wall-clock latency	10	2	0
Cost per task	6	6	0
Quality (mean score)	6	5	1
Total tokens processed	6	5	1

Why pi is the default

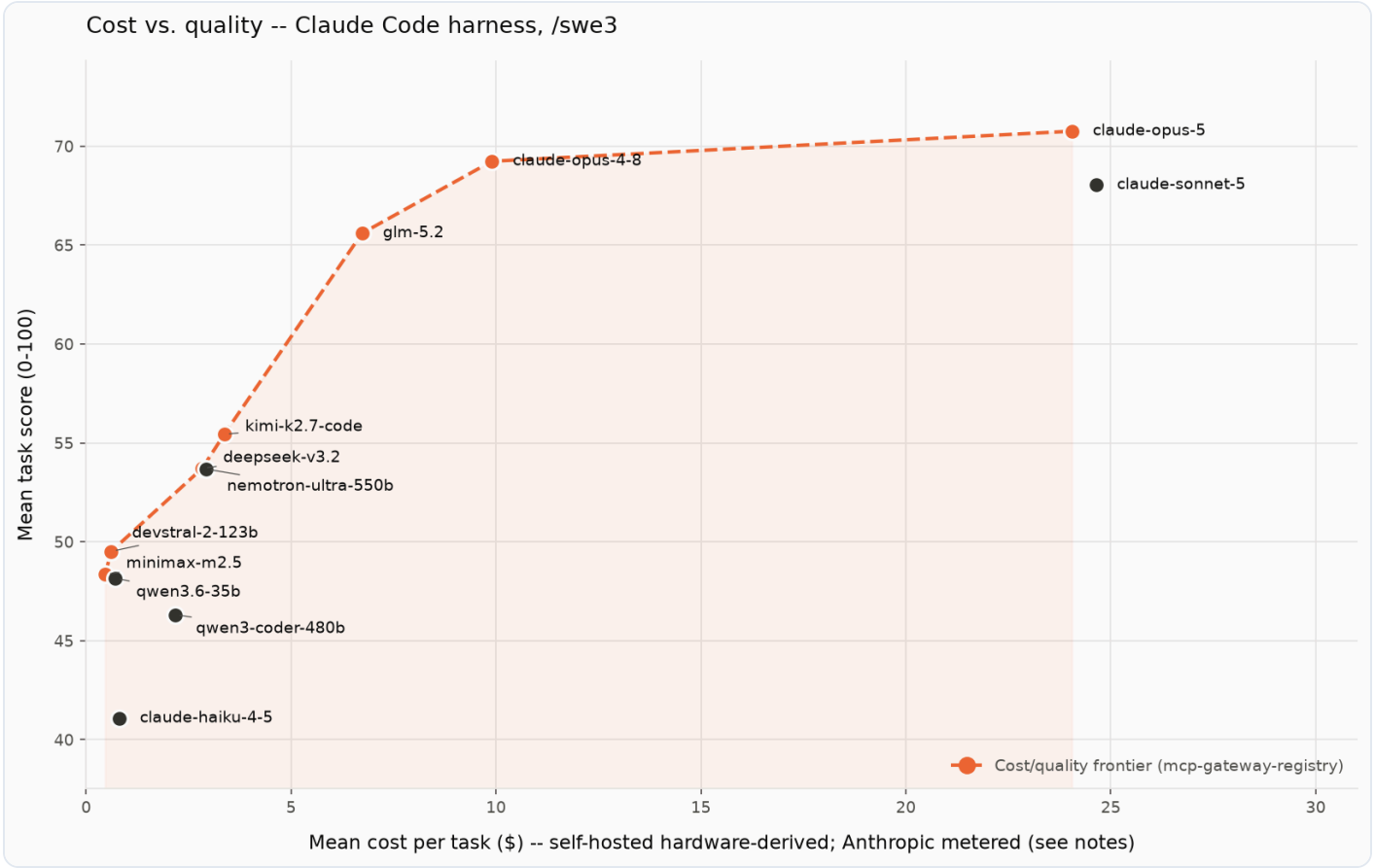
Its single-agent loop is consistently faster in wall-clock (no sub-agent fan-out to coordinate) and often cheaper -- for a developer at the terminal, the better default on latency and cost.

When to switch to Claude Code

Its multi-agent orchestration can lift quality on some models, at the price of more tokens, dollars, and time. Switch when a specific model scores meaningfully higher there and the task justifies the spend.

The same model can sit very differently under the two harnesses -- compare its row across the results tables and its bubble in each chart.

The same frontier under Claude Code (/swe3)



Same 14 models and 5 tasks, driven by Claude Code. Compare with the pi frontier: Claude Code pushes far more tokens per run, so self-hosted points move right (costlier) and wall-clock lengthens -- which is why the harness, not just the model, is a cost lever.

If you put both hosting paths on one axis

Combined frontier (pi /swe3)	Hosting	Score	$/task
claude-haiku-4-5	Bedrock	47.1	$0.64
qwen3.6-35b	self-hosted	52.3	$0.87
claude-sonnet-5	Bedrock	66.5	$3.81
claude-opus-5	Bedrock	75.7	$8.28

Bedrock's Anthropic ladder dominates almost every slot. qwen3.6-35b is the lone open-weight model that survives the combined frontier -- the one to keep in a Bedrock-first shop (with a 4/5 reliability asterisk).

Directional only: metered Bedrock bills and hardware-derived self-hosted costs are not comparable as raw dollars. Reserved/committed GPU pricing pulls more open-weight models onto the frontier (see the methodology section).

METHODOLOGY DEEP-DIVE

04

Methodology

How a run works end to end, how cost is derived (GPU-seconds × $/second), and where models are strong or weak.

From long-horizon task to a cost/quality point

1 · Long-horizon task

Point the agent at a real repo + issue. It drives an open-ended tool-use loop over **~50-250 turns** (up to 300+), ~10 min to over an hour per task.

2 · Prefill-heavy tokens

Each turn replays the growing transcript as fresh input and emits a tiny edit -- **~150:1 to ~660:1** input:output. Measured, not assumed.

3 · Six artifacts

Lands **github-issue, LLD, review, testing** + the implemented change (**patch.diff, implementation.md**) on disk.

4 · Judge the quality

An independent judge (**codex exec, GPT-5.6-sol**, high effort) scores each artifact 0-100 on **completeness, correctness, specificity, risk-awareness** against a fixed rubric, read-only against a fresh clone.

5 · Marry with throughput

Multiply each run's real tokens by the model's measured tokens/sec on its GPU to get a **hardware-derived cost per task** -- quality × cost on the same real work.

= One frontier point

Every model becomes a (cost, quality) point; the non-dominated set is the frontier a buyer chooses from.

Two non-comparable cost bases

Metered (Bedrock)

A hosted API's real per-token bill, summed over the run. Benefits from prompt caching -- cache-read tokens billed at ~10%.

Hardware-derived (self-hosted)

A rented GPU has no per-token bill. Cost = **GPU-seconds × $/second** = (tokens ÷ measured throughput) × instance $/hr. Not wall-clock × $/hr -- that would charge idle agent-thinking time.

Compare within a hosting basis. Cross-hosting dollars are an order-of-magnitude result, not an exact tie -- prompt caching is credited very differently on each side.

Full cost methodology →

What actually lowers self-hosted cost

Concurrency, up to the KV wall

Amortize the fixed $/hr across more sessions -- until the KV cache saturates. On a big model that wall comes early (GLM-5.2 saturates at ~c=5 on 8×H200).

Model-to-box fit

A small-active MoE that fits on half the box serves far more concurrent sessions per dollar than a model that fills it.

Reserved / committed pricing

The leaderboard now uses committed-capacity rates (Capacity Reservation / 1-year RI). On-demand would be higher; spot or higher utilization lower -- each shifts the frontier.

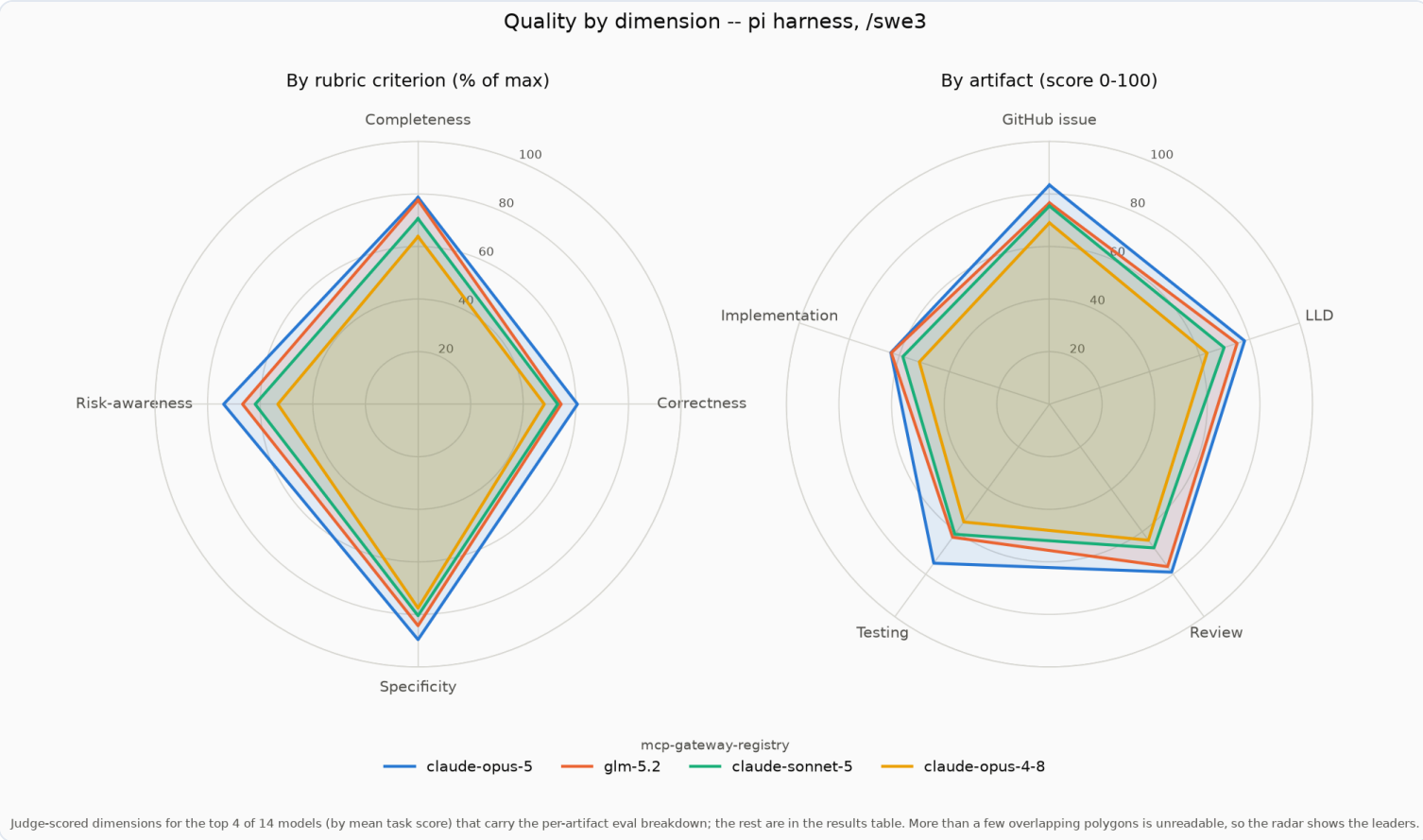
Caching is **already baked into the throughput** (98-99% prefix-cache hit rate raises tokens/sec) -- so it shows up as a lower rate, not a separate discount. The lever is **KV headroom**, which trades against context window and therefore accuracy: no free lunch.

Throughput & cost per model (vLLM sweep)

Model	Instance	$/hr	Peak gen tok/s	Blended $/1M	$/task
minimax-m2.5	p5en (TP=4)	$11.08	300	$0.06	$0.07
qwen3.6-35b	g6e.12xlarge	$4.53	145	$0.10	$0.14
qwen3-coder-30b	g6e.12xlarge	$4.53	67	$0.06	$0.17
devstral-2-123b	p5en (TP=4)	$11.08	128	$0.11	$0.61
nemotron-ultra-550b	p5en.48xlarge	$22.15	244	$0.15	$0.61
qwen3-coder-480b	p5en (TP=4)	$11.08	49	$0.19	$0.81
deepseek-v3.2	p5en.48xlarge	$22.15	173	$0.21	$1.66
kimi-k2.7-code	p5en.48xlarge	$22.15	274	$0.29	$2.34
glm-5.2	p5en.48xlarge	$22.15	190	$0.39	$3.12
gemma-4-31b	g6e.12xlarge	$4.53	31	$0.32	$0.45

Blended $/1M and $/task are hardware-derived at each model's true instance rate and peak-efficient concurrency (synthetic throughput sweep, 200K window, committed-capacity pricing). Small-active MoEs (minimax-m2.5) and the g6e models dominate on cost per token; the full-box H200 rate drives up the large models' per-task cost.

Where models are strong or weak



The judge scores four criteria (complete, correct, specific, risk-aware) across the artifacts. Every model dips hardest on **implementation** and **correctness** -- landing working code is the hard part.

05

Get started

Reproduce the frontier on your own repos and models -- nothing here is specific to the example repository.

Where this is headed

The frontier is the lookup table for a **cost-aware harness that routes each task to the right model** -- plan with a frontier model, execute with a cheaper workhorse, switch back up when a run needs it -- so a developer gets frontier-quality results at a fraction of the cost without managing model selection.

1. Point at a repo

Any GitHub URL + a task description. The example repo is the example, not the contract.

2. Pick a path

Anthropic on Bedrock, open-weight on Bedrock (LiteLLM), or self-hosted vLLM.

3. Read your frontier

Run both harnesses, score with the judge, choose on cost vs quality.

GitHub Repository **Full /swe3 results**

Scan to explore

The full harness, benchmarks, results, and methodology -- open source under MIT-0.

github.com/aarora79/agentic-coding-harness-benchmarks

Scan for the repository